

Cómo empezar un proyecto con este sistema

Guía rápida, paso a paso: dónde va cada carpeta, qué comando se ejecuta primero, y cómo se encadenan las fases hasta tener el MVP construido.

PRD.md → bootstrap.py → entrevista → SPEC.md
→ plan de fases → TASK-F0 ... TASK-Fn

0 ANTES DE EMPEZAR — QUÉ NECESITAS TENER A MANO

Cinco cosas, nada más:

- Una carpeta vacía para el proyecto nuevo (una por cliente/proyecto).
- Un documento de negocio** — PRD.md, MVP.md o brief.md. No hace falta que sea largo: con un párrafo de problema, usuario y funcionalidades imprescindibles, el sistema ya puede arrancar.
- Las plantillas maestras del harness:** INICIO_PROYECTO.md, SECURITY.md, AEO_GEO_SEO.md, UI_UX_EXCLUSIVA.md, SKILLS_MCP.md, TASK_TEMPLATE.md, TASK_LITE_TEMPLATE.md, QUICKSTART_LITE.md.
- Los dos scripts:** bootstrap.py y task_generator.py (usa la versión corregida, no la original).
- Python 3** instalado, y un agente de IA con acceso a esa carpeta (terminal local + DeepSeek/OpenCode, Claude Code, etc.).

LA IDEA EN UNA FRASE

Tú pones el PRD y las plantillas; el sistema hace preguntas, escribe la especificación, la trocea en fases pequeñas, y genera una tarea concreta (TASK-Fx.md) para que el agente ejecute una fase cada vez — nunca "todo de golpe".

1 DÓNDE VA CADA CARPETA

Antes de ejecutar nada, la carpeta de tu proyecto nuevo debe tener solo esto:

```
mi-proyecto/
├── PRD.md                <- tu documento de negocio, en la raíz
├── docs/                 <- las plantillas maestras, TAL CUAL, sin tocar
│   ├── INICIO_PROYECTO.md
│   ├── SECURITY.md
│   ├── AEO_GEO_SEO.md
│   ├── UI_UX_EXCLUSIVA.md
│   ├── SKILLS_MCP.md
│   ├── TASK_TEMPLATE.md
│   ├── TASK_LITE_TEMPLATE.md
│   └── QUICKSTART_LITE.md
```

Los dos scripts (bootstrap.py y task_generator.py) van sueltos en la raíz de mi-proyecto/, al mismo nivel que PRD.md — no dentro de docs/.

EL ERROR MÁS TÍPICO

Poner las plantillas sueltas en la raíz en vez de dentro de docs/. bootstrap.py busca la carpeta docs/ específicamente para copiar desde ahí — si no la encuentra, se detiene con un error.

2 EL FLUJO COMPLETO, DE UN VISTAZO

Hay dos tipos de fase, con dos códigos distintos — no se mezclan nunca:

CÓDIGO	QUÉ ES	SE RESUELVE...
M0–M3	Fases del proceso. Siempre las mismas 4, en cualquier proyecto: leer el PRD, entrevista técnica, escribir SPEC.md, planificar las fases.	Hablando con el agente / leyendo INICIO_PROYECTO.md. No generan TASK-Mx.md.

F0–Fn	Fases de ejecución. Específicas de tu proyecto (modelo de datos, backend, frontend...), se definen en M3 a partir de SPEC.md.	Cada una con su propio TASK-Fx.md, generado por task_generator.py.
-------	---	--

M0 Bootstrap → M1 Entrevista → M2 SPEC.md → M3 Plan de fases

↓
F0 → F1 → F2 → ... → Fn
(por cada F: generar → ejecutar → validar → cerrar)
se repite hasta que no quede ninguna F pendiente

Todo lo que el sistema va aprendiendo o decidiendo vive en cuatro archivos, que se generan solos y se van actualizando — tú casi nunca los escribes a mano desde cero:

ARCHIVO	PARA QUÉ SIRVE
CONTEXT.md	Resumen vivo del proyecto: qué es, para quién, decisiones ya tomadas.
SPEC.md	La especificación técnica completa (se escribe en M2).
PROGRESS.md	El estado de todas las fases: qué está cerrado y qué falta.
DECISIONS.md	Por qué se eligió cada cosa (base de datos, stack, etc.).

3 PASO A PASO

1. Crea la carpeta y coloca los archivos

Sigue el árbol de la sección 1: PRD.md en la raíz, plantillas dentro de docs/, los dos scripts sueltos en la raíz.

2. Ejecuta bootstrap.py (fase M0)

```
cd mi-proyecto
python3 bootstrap.py
```

Esto hace cuatro cosas automáticamente:

```
mi-proyecto/
├── src/ tests/ infra/      <- carpetas creadas
├── SECURITY.md
├── AEO_GEO_SEO.md
├── UI_UX_EXCLUSIVA.md
├── SKILLS_MCP.md
├── TASK_TEMPLATE.md
├── TASK_LITE_TEMPLATE.md
├── QUICKSTART_LITE.md
├── INICIO_PROYECTO.md     <- copiadas a la raíz
├── CONTEXT.md             <- generado: resumen del PRD
├── PROGRESS.md            <- generado: solo M0-M3
└── DECISIONS.md          <- generado
```

REVISAS ESTO ANTES DE SEGUIR

Abre CONTEXT.md: si el PRD no tenía secciones claras de "Problema" o "Usuarios", el script lo marca con ⚠ Sin confirmar. Complétalo a mano antes de aprobar nada.

3. Entrevista técnica (fase M1)

Dale al agente de IA acceso a la carpeta y pídele que lea `CONTEXT.md`. El propio archivo trae la instrucción de arranque: el agente empezará a hacerte preguntas (stack, base de datos, autenticación, servidores, integraciones, SEO/AEO/GEO...) siguiendo `INICIO_PROYECTO.md`. Con tus respuestas rellena `SECURITY.md` y `AEO_GEO_SEO.md`.

Cuando termine, marca M1 como [x] Listo en `PROGRESS.md`.

4. SPEC.md (fase M2)

Con el PRD y la entrevista ya cerrados, el agente redacta `SPEC.md`: arquitectura, modelo de datos, endpoints, criterios de aceptación. **Tú lo apruebas explícitamente** antes de seguir — es el único punto de control obligatorio antes de que se planifique ninguna fase de código.

5. Plan de fases (fase M3)

El agente descompone `SPEC.md` en fases pequeñas (F0, F1, F2...) siguiendo la tabla de ejemplo de `INICIO_PROYECTO.md`. Esa tabla se pega tal cual dentro de `PROGRESS.md`, en la sección ya preparada para ello — sin cambiar el orden de columnas.

```
## Fases de Ejecución del Proyecto (F0-Fn)

F0
  Objetivo:    Bootstrap del repo, tooling, CI básico
  Entregable:  Repo funcionando
  Depende de:  SPEC aprobado
  Estado:      [ ] Pendiente

F1
  Objetivo:    Modelo de datos + migraciones
  Entregable:  BBDD creada y versionada con RLS
  Depende de:  F0
  Estado:      [ ] Pendiente

... (F2, F3... siguen el mismo patrón)
```

6. Generar y cerrar cada tarea (bucle, una vez por cada F)

A partir de aquí, repites este ciclo hasta que no quede ninguna fase F pendiente:

```
python3 task_generator.py
```

El script detecta solo la primera fase F pendiente, mira si toca seguridad, visibilidad (SEO/AEO/GEO) o UI/UX, y genera `TASK-F<N>.md` con los checklists reales ya insertados. Pásale ese archivo al agente: ejecuta la fase, corre los tests, y cierra la tarea marcando F<N> como [x] Listo en `PROGRESS.md`. Vuelve a lanzar el comando para la siguiente.

4 CHULETA DE COMANDOS

```
# 1. una sola vez, al principio
python3 bootstrap.py

# 2. genera la siguiente tarea pendiente (detecta la fase sola)
python3 task_generator.py

# 3. generar una fase concreta a propósito
python3 task_generator.py --phase F3

# 4. forzar la plantilla reducida (proyecto pequeño / bajo riesgo)
python3 task_generator.py --lite
```

```
# 5. si mi-proyecto/ no es la carpeta actual
python3 task_generator.py --dir /ruta/a/mi-proyecto
```

5 PREGUNTAS DE TORPE

¿Puedo tocar los archivos de docs/?

No hace falta. Son la plantilla maestra; `bootstrap.py` copia una versión activa a la raíz, que es la que se rellena y se modifica proyecto a proyecto.

Ejecuté `task_generator.py` y dice que faltan fases M

Es normal recién hecho el bootstrap: todavía no existe la tabla `F0-Fn` porque `SPEC.md` no se ha escrito. Completa M1-M3 primero (pasos 3 a 5).

¿Puedo saltarme la entrevista técnica si tengo prisa?

No. Es la única forma de que `SECURITY.md` se rellene con decisiones reales en vez de placeholders. Si el proyecto es realmente pequeño, usa Modo Lite (`--lite`) en vez de saltarte pasos.

¿Dónde anoto por qué elegí Postgres en vez de Firebase?

En `DECISIONS.md`. Cualquier decisión técnica relevante se registra ahí, con la alternativa descartada y el motivo.

¿Qué hago cuando ya no queda ninguna fase F pendiente?

`task_generator.py` te lo dice explícitamente ("no queda ninguna fase de ejecución pendiente"). Repasa el checklist de seguridad completo (`SECURITY.md` §2.1–2.10) antes de dar el MVP por cerrado.

REGLA DE ORO, RESUMIDA

No se escribe una sola línea de código de producción hasta tener: PRD leído (M0), entrevista respondida (M1) y `SPEC.md` aprobado por ti (M2). Todo lo demás son fases F, una `TASK-Fx.md` cada vez.